

chapter10

제10장 모델 배포와 CI/CD

9장에서 champion 별칭까지 확정된 모델은 아직 실습 스크립트 안에서만 로드되었다. 실제 이용자는 스크립트를 실행하지 않는다 — 모델은 API 뒤에 놓여 요청을 받아야 하고, 그 API는 어느 서버에서든 같은 형태로 실행되도록 포장되어야 하고, 포장과 교체는 사람의 손작업이 아니라 검증 파이프라인을 통과해야 한다. 이 장은 그 세 층을 차례로 세운다: 예측을 언제 만들 것인가(배치/온라인 추론), 새 버전을 어떻게 교체할 것인가(일괄·카나리·shadow·blue-green), 서빙 API를 어떻게 만들고(FastAPI) 포장하고(Docker) 검증하는가(GitHub Actions). 9장 예고 그대로, 실습(★★★★☆)의 서빙 코드가 로드하는 것이 바로 `models:/complaint_daily_forecaster@champion` 이다 — 별칭 하나로 이어진 레지스트리와 서빙 사이에서, 호스트와 컨테이너가 같은 입력에 같은 예측(6.0)을 반환하는 것까지 실측으로 확인한다.

학습 목표

모델 배포 전략을 비교한다.

FastAPI와 Docker로 간단한 모델 API를 만든다.

CI/CD 파이프라인의 기본 검증 단계를 이해한다.

요구사항에서 배포 전략·롤백 트리거·CI 게이트 배치·기동 대기 시간을 고르고, 각 값의 근거와 그 설계가 깨지는 조건을 적는다.

10.1 배치 추론과 온라인 추론

학습 목표

예측을 “미리 만드는가, 요청 순간 만드는가”의 두 방식과 판단 기준을 안다.

실패의 형태가 두 방식에서 어떻게 다른지 구분한다.

직관: 예측은 언제 계산되는가

모델이 있다고 예측이 이용자에게 도달하는 것은 아니다 — 누군가 어느 시점에 모델을 실행해야 한다. 그 시점의 선택지가 둘이다. **배치 추론**은 예측을 미리 만들어 저장해 둔다: 매일 밤 전체 지역구의 내일 민원 건수를 한꺼번에 예측해 테이블에 넣고, 낮의 조회는 계산이 아니라 저장된 값의 lookup이다. **온라인 추론**은 요청이 도착한 순간 계산한다: 이용자가 입력을 보내면 API가 모델을 실행해 그 자리에서 답을 만든다. 7장(배치 확정 집계)과 8장(온라인 스토어 조회)에서 본 데이터의 두 처리 방식이 예측에서도 반복되는 것이다.

선택 기준은 유행이 아니라 두 질문이다. 첫째, **요청 시점에만 알 수 있는 입력이 필요한가?** 이용자가 방금 입력한 값·현재 위치처럼 미리 알 수 없는 입력이 있으면 배치로는 만들 수 없다. 둘째, **예측이 소비될 지점을 미리 열거할 수 있는가?** “서울 25개 자치구의 내일 예측”처럼 대상이 유한하고 열거 가능하면 미리 다 만들어 둘 수 있고, “임의의 입력 조합”이면 불가능하다.

표 10.1 배치 추론과 온라인 추론(민원 예측 시나리오)

비교 축	배치 추론	온라인 추론
계산 시점	정해진 주기(예: 매일 밤)	요청 도착 순간
응답 지연	조회만 하므로 빠름	모델 실행 시간이 요청 경로에 들어옴
입력 조건	계산 시점에 아는 입력만	요청 시점의 입력 사용 가능
실패의 형태	낮은 예측을 서빙 (계산 실패가 조용히 지나감)	응답 불가 (서버 장애가 즉시 드러남)
인프라	스케줄러 + 저장소(7장의 연장)	상시 서버·API·확장(이 장)
어울리는 과제	대상 열거 가능·주기 갱신으로 충분	요청별 입력·즉시성 요구

실패의 형태 행이 가장 중요하다. 배치의 실패는 조용하다 — 밤 배치가 실패해도 낮의 조회는 어제 값을 아무 오류 없이 반환한다(7장의 `_SUCCESS` 마커가 정확히 이 사고를 막는 장치였다). 온라인의 실패는 즉시 드러난다 — 서버가 중단되면 모든 요청이 즉시 실패한다. 어느 쪽이 나쁜가는 과제에 따라 다르지만, **어느 쪽의 실패인지 알고 설계하는 것이** 운영의 시작이다.

두 방식의 구분은 도메인을 가리지 않는다. 민간의 추천 서비스도 매일 밤 전체 사용자의 추천 후보를 미리 계산해 두고 낮에는 저장된 값을 조회한다 — 전 지역구의 내일 예측을 미리 만들어 두는 것과 구조가 같다. 다른 것은 규모다. 대상이 수천만이면 배치 한 번의 계산량이 커지는 대신 요청마다 계산하지 않아 아끼는 몫도 그만큼 커지므로, 규모가 커질수록 배치를 선택할 유인이 커진다. 판단 기준(입력의 가용 시점, 대상의 열거 가능성)은 규모와 무관하게 그대로다.

반대 논거: 이 문제에 온라인이 필요한가

일 단위 민원 건수 예측이라면 사실 배치로 충분하다 — 대상(자치구)은 열거 가능하고, 피처(전일 건수)는 하루 한 번 확정되며, 이용자가 1분마다 새 예측을 요구할 이유가 없다. 실습이 그럼에도 온라인 API를 세우는 이유를 명시한다: 서빙 계약(/health·predict·입력 검증), 컨테이너 포장, CI 게이트라는 **배포의 규율**은 온라인 서빙에서 전부 등장하므로 관찰 대상으로 그쪽을 택한 것이다. 반대로 배치로 충분한 문제에 상시 API와 오토스케일링을 구축하면 비용과 장애 지점이 늘어나는 과잉 설계가 된다. 8장 온라인 스토어가 제시한 절충(배치로 만든 값을 온라인으로 조회)은 예측에도 그대로 적용된다: 배치 추론 + 결과의 온라인 lookup이 실무의 큰 몫을 차지한다.

공공 사례

재난지원금 자격 예측처럼 “전 국민 대상, 기준일 확정”인 과제는 전형적 배치다 — 대상이 열거 가능하고 입력이 기준일에 확정된다. 반면 민원 챗봇의 다음 문장 분류는 온라인일 수밖에 없다 — 입력이 방금 도착했기 때문이다. 발주 문서에서 “실시간 AI”라는 표현을 확인하면 물어야 한다: 요청 시점에만 아는 입력이 실제로 있는가, 아니면 배치+조회로 동등한 응답성을 더 낮은 비용과 위험으로 제공할 수 있는가. ### 핵심 정리 - 배치=미리 계산해 저장(조회는 lookup), 온라인=요청 순간 계산 — 판단 기준은 입력의 가용 시점과 대상의 열거 가능성이다. - 실패의 형태가 다르다: 배치는 낮은 예측을 조용히 서빙하고, 온라인은 시끄럽게 응답 불가가 된다. - 배치로 충분한 문제에 온라인 인프라를 세우는 것도 과잉 설계다 — “실시간”이 정말 필요한지 먼저 묻는다.

10.2 일괄, 카나리, shadow, blue-green 배포

학습 목표

네 배포 전략을 실패 반경·롤백(rollback) 경로·비용으로 비교한다.
9장 별칭 이동(모델 층)과 이 장 컨테이너 교체(서비스 층)의 관계를 이해한다.
롤백 트리거를 지표·임계·관측 창의 세 요소로 적는다.

직관: 확신은 배포 전에 완전해지지 않는다

새 버전이 옳다는 확신은 배포 전에 완전해질 수 없다 — 테스트가 아무리 많아도, 실제 트래픽·실제 데이터가 만드는 상황의 전부를 미리 볼 수는 없다(10.4에서 이 문장의 실측 증거를 확인한다: 모든 사전 게이트를 통과한 코드가 컨테이너에서 즉시 실패했다). 그래서 배포 전략은 전부 한 질문의 답이다 — **틀렸을 때 피해가 어디까지 번지는가(실패 반경, blast radius)를 어떻게 제한하는가.**

일괄(recreate): 구버전을 내리고 신버전을 올린다. 가장 단순하고, 실패 반경은 100%다 — 신버전이 틀리면 전체 이용자가 영향을 받는다.

blue-green: 두 환경(blue=현재, green=신규)을 나란히 두고 트래픽 스위치를 한 번에 넘긴다. 반경은 여전히 100%지만 **롤백이 스위치 한 번**이다 — 구환경이 살아 있기 때문이다. 비용은 환경 2벌.

카나리(canary): 신버전에 트래픽의 일부(예: 5%)만 보내고, 이상 없으면 점증한다. 실패 반경이 5%로 시작한다. 대가는 트래픽을 비율에 따라 분할하는 라우팅 인프라와, “이상 없음”을 판정할 지표(11장)다.

shadow: 실제 요청을 신버전에 **복제**하되 응답은 구버전 것만 사용한다. 실패 반경 0 — 신버전이 무엇을 답하든 이용자에게 닿지 않는다. 대가는 계산 자원 2배와, 응답을 쓰지 않으므로 “이용자 반응”은 관찰할 수 없다는 한계다.

표 10.2 배포 전략 비교(정본 산출물)

전략	교체 방식	실패 반경	롤백 경로	추가 비용	어울리는 상황
일괄	전량 즉시 교체	100%	재배포(느림)	없음	내부 도구, 정지 허용
blue-green	환경 2별 + 스위치	100%(순간)	스위치 되돌림(즉시)	환경 2별	빠른 복귀가 최우선
카나리	트래픽 비율 점증	시작 비율만큼	비율 0으로 축소	라우팅 + 판정 지표	대규모·외부 대상 서비스
shadow	요청 복제, 응답 미사용	0	해당 없음(노출 없음)	계산 2배	교체 전 후보 검증

최신 이론·현장: 네 전략은 어디서 왔는가

이 네 이름은 교체가 지어낸 것이 아니라 지속적 배포(continuous delivery) 실무에서 굳어진 패턴이다. blue-green과 canary의 정의는 이 실무 문헌에서 그대로 가져올 수 있다. Danilo Sato는 카나리 릴리스를 “새 버전을 전체 인프라에 한 번에 내보내기 전에, 소수의 이용자에게 먼저 점진적으로 배포해 위험을 줄이는 기법”으로 정의하고 — blue-green과 마찬가지로 트래픽이 없는 일부 인프라에 신버전을 먼저 배포한 뒤, 만족스러우면 선별된 사용자부터 라우팅을 넘긴다고 설명한다. 표 10.2의 “실패 반경을 시작 비율만큼 제한한다”가 이 서술의 다른 표현이다 — 다만 완전한 격리를 전제한 근사임을 SRE 원문이 짚는다: 카나리와 기존 버전이 백엔드·저장소·네트워크를 공유하면 카나리의 문제가 기존 버전에까지 번져 반경이 시작 비율을 넘어설 수 있다. Google의 SRE 워크북은 카나리를 배포 안전성(deployment safety)의 문제로 다루면서, 카나리 설계의 핵심 결정으로 **얼마나 노출할 것인가(카나리 규모)**와 **무엇으로 정상/이상을 판정할 것인가(평가 지표)**를 함께 놓는다. 표 10.2의 “추가 비용” 열에 “라우팅 + 판정 지표”라 적은 것이 이 둘이며, 그 판정 지표를 실제로 무엇으로 삼을지는 11장(모니터링)의 몫으로 넘어간다.

적용 자격을 명시한다: 위 출처들(Fowler의 Blue-Green, Sato의 Canary Release, Google SRE)은 **일반 소프트웨어 배포** 도메인의 문헌으로, 공공 ML 서빙의 직접 선례가 아니다 — 패턴의 어휘와 “위험을 노출 범위로 제한한다”는 논리를 **방법론으로 차용**하는 것이다. ML에 특화된 관점은 Thoughtworks의 CD4ML(Continuous Delivery for Machine Learning) 문헌이 제공한다 — 모델 서빙·모델 배포·ML에서의 테스트를 CD 파이프라인의 구성요소로 엮은 종단간 참조로, 이 장의 “서빙→포장→검증” 골격과 같은 문제를 다룬다. 다만 CD4ML의 예제는 판매 예측이라는 **민간 ML**이므로 이 역시 방법론 차용 자격이다(대상은 민간, 우리는 공공). 이 장이 CD4ML과 다른 지점 — 시민 대우의 형평성 — 이 바로 다음 절의 적용 문제다.

민간→공공 적용: 트래픽 분할 실험에서 후보 모델 로그 평가로

민간 배포 문화의 중심에는 **트래픽을 분할해 이용자 집단별로 서로 다른 버전을 제공하는** 방식이 있다. 카나리는 신버전에 트래픽을 5%→50%→100%로 점진 전환하고, A/B 릴리스는 이용자를 무작위 두 집단으로 나눠 각 집단에 다른 버전을 서비스한다. 이 둘의 목적은 사실 다르다 — 카나리는 신버전이 기존을 망가뜨리지 않는지(회귀·장애)를 제한 노출로 확인하는 안전장치이고, A/B는 어느 버전이 목표 지표에서 더 나은지를 가설로 검정하는 실험이다(Sato는 기술 구현이

비슷하더라도 이 둘을 혼동하지 말라고 명시한다). 그럼에도 공공 적용의 관점에서 둘은 한 성질을 공유한다 — **같은 시각에 시민마다 다른 버전이 응답한다**는 것. 이 특성 때문에 공공 분야에서는 카나리와 A/B 적용이 제한된다. 시민 A는 신모델의 판정을, 시민 B는 구모델의 판정을 받게 하는 무작위 분할은 “같은 상황의 시민을 우연히 다르게 대우”하는 것이 되어, 민원 배정·복지 판정처럼 권리에 영향을 주는 결정일수록 형평성과 법적 정당성 문제가 첨예해진다(1장 표 1.6, 16장 공정성).

공공에서는 **응답 경로를 champion 하나로 고정**해 모든 시민에게 같은 모델의 결과를 제공하고, challenger는 같은 입력을 받아 예측 결과를 로그에만 저장한다. 후보 모델의 결과는 승인 전까지 시민 서비스에 사용하지 않는다. 로그에 저장한 champion 6.0과 challenger 7.3636을 검수자가 비교한 뒤, 승인하면 별칭 이동과 재배포로 한 번에 전환한다.

표 10.2b 민간 배포 실험의 공공 적용

민간 방법	목적	공공 변형	왜 변형하는가
카나리(트래픽 점진 전환)	노출을 늘리며 실험트래픽에서 검증	승인 게이트 + 검수 후 일괄 전환	“일부만 신버전” 중간 상태가 시민 간 대우 차이를 만든다
A/B 릴리스(무작위 분할)	두 버전을 라이브로 동시 비교	shadow(응답 고정 + 로그 관찰)	시민 무작위 분할은 형평성·법적 정당성 문제
점진 전환 후 자동 승격	지표 자동 판정으로 롤아웃	사람 검수·승인 후 전환 (감사 기록)	공공 배포는 책임 주체와 승인 이력이 산출물

일괄 전환이 형평성 면에서 오히려 단순하다는 점에 유의한다 — 전환 전에는 전원이 구버전을, 전환 후에는 전원이 신버전을 받고, 그 사이에 “누구는 신버전, 누구는 구버전”인 구간이 없다. 민간이 위험을 줄이려 만든 중간 상태(비율 롤아웃)가 공공에서는 형평성 위험을 만드는 상태가 되므로, 위험을 노출 범위가 아니라 **관찰 경로(shadow)와 승인 게이트**로 제한하는 쪽으로 설계가 옮겨간다. 다만 이 제약은 권리에 영향을 주는 판정에 적용되는 것이지 카나리형 부분 노출을 일괄 금지하는 것은 아니다 — 공지·안내 UI·권리와 무관한 내부 기능처럼 시민 간 대우 차이가 문제되지 않는 영역에서는 카나리도 그대로 유효하다.

워크드 예제: shadow를 실측으로 — 반려된 후보의 두 번째 기회

9장에서 선형회귀 후보(v2)는 승격 반려되었다 — 절대 게이트는 통과했지만 champion 대비 개선이 없었다. 반려로 끝나는 것은 아니다: 훈련 오차 6쌍으로 내린 판정이므로, **실제 요청 위에서** 두 후보를 나란히 관찰할 가치가 남아 있다. 이것이 shadow의 자리다. 실습의 서빙 API는 v2에 challenger 별칭을 붙이고(표 9.3a의 검증 단계 관례 — 9장 실습에는 없던 별칭을 이 장의 재구성이 부여한다), 요청마다 champion으로 응답하면서 challenger의 예측을 조용히 로그로만 남긴다.

실측(전체는 실습 10.1): 입력 전일 건수 9(강남구, 8장 온라인 스토어 실측값)에 대해 —

이용자가 받은 응답: **6.0** (champion=v1, 평균 예측기 — 훈련 y 평균 36/6, 9장 검산 그대로)

로그에만 남은 값: challenger **7.3636** ($v2$ 선형회귀: $3.6818 + 0.4091 \times 9 = 81/22 + 81/22 = 162/22 = \mathbf{81/11}$ — 9장 표 9.2의 계수로 손 계산된다)

두 값의 차이(1.36)가 곧 shadow가 만드는 관찰 자료다 — 이런 로그가 쌓이고 실제 결과(당일 실측 건수)와 대조되면, “ $v2$ 가 실요청에서 정말 나온가”를 이용자 위험 0으로 판정할 수 있다. 반려 이력(9장)이 감사 기록이었다면, shadow는 반려 후보의 **재심 절차**다.

두 개의 교체 층: 별칭 이동과 컨테이너 교체

9장 끝에서 “별칭 이동이 배포 전략의 한 축”이라 예고했다. 나머지 한 축이 이 장에서 드러난다. 별칭 이동은 **모델 층**의 교체다 — 레지스트리에서 champion이 $v1$ 에서 $v2$ 로 옮겨진다. 그러나 실습의 서빙 앱은 모델을 **시작 시 1회** 로드한다(10.3) — 실행 중인 프로세스는 별칭이 어디로 옮겨졌든 자기가 로드해 둔 모델로 계속 응답한다. 즉 별칭 이동은 “다음에 로드하는 소비자부터” 유효하고, 그 “다음 로드”를 일으키는 행위가 **서비스 층**의 교체 — 재기동, 곧 배포다. 두 층이 만나야 교체가 완성된다: 별칭을 옮기고(모델 층), 새 컨테이너를 기동해 트래픽을 전환한다(서비스 층, 표 10.2의 전략들). 반대로 보면 롤백도 두 층이다 — 별칭을 되돌리는 것(9장)과 이전 이미지를 다시 기동하는 것(10.4)은 같은 목표의 다른 층 동작이다.

롤백 트리거: 되돌릴 경로가 있어도 기준이 없으면 되돌리지 않는다

표 10.2의 “롤백 경로” 열은 되돌리는 **방법**만 답한다. 배포 사고가 커지는 자리는 대개 방법이 없어서가 아니라 **언제 되돌릴지**를 정해 두지 않아서다 — 지표가 저하되는 동안 담당자들은 “관측을 연장하자”는 판단을 반복하고, 되돌림은 피해가 확산된 뒤에 이루어진다. 따라서 배포 문서에는 **지표, 임계값, 관측 기간**을 모두 명시한다. 하나라도 빠지면 실행 가능한 롤백 조건이 아니다.

지표는 층마다 다른 것을 고른다. 서비스 층은 오류율·응답 지연처럼 즉시 관측되는 값이고, 모델 층은 예측 분포의 이탈처럼 늦게 드러나는 값이다(무엇으로 어떻게 측정하는지는 11장의 몫이다). 이 장의 범위에서 바로 쓸 수 있는 트리거는 스모크다 — 배포 직후 알려진 입력 9에 알려진 출력 6.0이 나오지 않으면 그 자리에서 이전 이미지로 되돌린다. 임계와 관측 창은 되돌림의 비용에서 역산한다. 되돌림 비용이 낮으면(이전 이미지 재기동) 창을 짧게 설정해 빨리 되돌려도 손실이 적고, 되돌림 비용이 높으면(저장소 스키마 변경을 동반하는 교체) 창을 늘려 오탐을 줄인다. 공공에서는 한 항목이 더 추가된다 — 자동으로 되돌릴 것인가 사람이 판단할 것인가, 그리고 되돌린 사실과 그 근거를 어디에 남길 것인가가 승인 이력과 함께 산출물이 된다.

민간에서는 이 되돌림의 속도 자체를 지표로 관리한다. 배포 실패에서 정상 복귀까지 걸린 시간(MTTR, Mean Time To Recovery)과 배포 중 장애를 일으킨 비율(변경 실패율, change failure rate)은 DORA(DevOps Research and Assessment)가 정리한 표준 지표이고, blue-green과 카나리를 선호하는 이유도 이 둘을 줄이기 때문이다. 공공에서 같은 값을 측정하는 목적은 다르다 — 개발 조직의 평가 지표가 아니라 “장애가 나면 몇 분 안에 원상 복구되는가”라는 서비스 수준 약속의 근거다.

공공 사례

민원 접수 시스템의 분류 모델을 일괄 교체했다가 오분류가 전면 확산되면, 그것은 “일부 이용자 불편”이 아니라 행정 서비스 전체의 장애이자 감사 대상이다. 공공 서비스에서 실패 반경 제한은 성능 최적화가 아니라 **책임의 요구사항**이다 — 발주·검수 문서에 “신규 모델은 shadow 관찰과 승인 게이트를 거쳐 전환하며, 즉시 롤백 경로를 갖춘다”가 명시되어야 하는 이유다(권리에 영향을 주는 판정에서는 시민 간 대우 차이를 만드는 카나리형 부분 노출 대신 shadow+ 일괄 전환을 기본으로 삼는다 — 앞의 “민간→공공 적용” 절). 표 10.2가 그 조항의 어휘를 제공한다. ### 핵심 정리 - 배포 전략은 전부 실패 반경의 제한 방식이다: 일괄 100%, blue-green 100%+ 즉시 복귀, 카나리 비율 제한, shadow 반경 0. - shadow 실측: 응답은 champion 6.0, 로그에는 challenger 7.3636(=81/11, 손 계산) — 반려 후보를 이용자 위험 없이 재평가한다. - 교체는 두 층이다: 별칭 이동(모델 층)은 다음 로드부터 유효하고, 그 로드를 일으키는 재기동·트래픽 전환(서비스 층)이 배포다. - 롤백 경로와 롤백 트리거는 다른 것이다 — 지표·임계·관측 창을 다 적어야 되돌림이 절차가 되고, 임계와 창은 되돌림의 대가에서 역산한다.

10.3 FastAPI 모델 서빙

학습 목표

서빙 API의 최소 계약 세 가지(신원 응답·예측·입력 검증)를 구현 수준에서 안다.
champion 별칭 로드가 서빙 코드에서 어떻게 나타나는지 실측으로 확인한다.

직관: API는 모델에 계약을 입힌다

스크립트 안의 모델은 호출자의 올바른 사용에 의존할 수밖에 없다. API는 그 기대를 **계약**으로 바꾼다. 실습 서빙 앱(FastAPI 0.139.0 + uvicorn 0.50.2)의 계약은 세 조각이다.

신원 응답(/health): 살아 있는가만이 아니라 **무엇을 서빙 중인가**를 답한다 — 모델 이름·버전·로드 출처. 9장의 감사 질문 “지금 운영 중인 모델은 정확히 무엇인가”가 여기서 API 한 번의 호출로 답해진다.

예측(/predict): 입력(지역구 코드, 전일 건수)을 받아 예측을 반환한다 — 응답에도 모델 신원이 함께 실린다(어느 버전이 만든 예측인지가 예측값만큼 중요하다).

입력 검증: 스키마에 어긋난 입력은 모델에 닿기 전에 **422**로 거절된다. `lawd_cd`는 5자리 숫자(7장 표준코드), `x_prev_count`는 0 이상 정수(8장 피처 정의) — pydantic 선언이 곧 기계가 집행하는 스키마 계약이며, 2장에서 본 “스키마 위반” 위험이 서빙 입구에서 차단되는 지점이다.

모델 로드는 **시작 시 1회**다(FastAPI lifespan). 요청마다 로드하면 로드 지연과 레지스트리 장애가 모두 요청 경로로 들어온다 — 시작 시 로드는 “무겁고 위험한 일은 기동 시점에, 요청 경로에는 계산만”이라는 서빙의 기본 규율이다. 그 대가로 10.2에서 확인한 성질이 발생한다: 실행 중인 프로세스는 별칭 이동을 인지하지 못한다.

핵심 코드

9장 예고의 실체 — 서버가 레지스트리를 만나는 지점은 이 몇 줄이 전부다.

```
model = mlflow.pyfunc.load_model("models:/complaint_daily_forecaster@champion")
mv = MlflowClient().get_model_version_by_alias("complaint_daily_forecaster", "champion")
app.state.identity = {"model_name": ..., "model_version": str(mv.version)} # /health가 답할 신원
```

전체 코드는 *practice/chapter10/code/10-1-model-api/app.py* 참고

버전 번호가 아니라 별칭으로 로드하므로, 다음 승격(별칭 이동) 때 이 코드는 한 줄도 바뀌지 않는다 — 9.3에서 예고한 소비 방식 그대로다. 신원은 로드와 별도로 레지스트리에 조회해 보관한다: “별칭이 지금 가리키는 버전”을 응답에 실을 수 있어야 /health가 감사 질문에 답한다.

워크드 예제: 서버 실측

호스트에서 uvicorn으로 앱을 띄우고(포트 8010) 호출한 실측이다(*ch10_deploy_report.json*).

표 10.3 서버 API 실측(실제 실행)

호출	응답(실측)	검산·근거
GET /health	status=ok, model_name=complaint_daily_forecaster, model_version=1 , model_source=registry-alias, shadow_enabled=true	champion=v1 — 9장 승격 판정과 일치
POST /predict {lawd_cd: 11680, x_prev_count: 9}	forecast=6.0 + 모델 신원	v1은 평균 예측기 — 훈련 y 평균 36/6=6.0(9장 검산 재현)
POST /predict {x_prev_count: -1}	422 (모델 미도달)	스키마 계약: 음수 건수는 입력일 수 없다
(shadow 로그)	champion 6.0 / challenger 7.3636	10.2의 손 검산(81/11)

입력 9가 응답 6.0에 반영되지 않는 것은 9장에서 정의한 champion(v1)의 동작이다 — 평균 예측기는 입력과 무관하게 같은 값을 낸다. 서버 계층에서는 **알려진 입력 → 알려진 출력**을 확인할 수 있으므로 *pytest·호스트·컨테이너·CI*의 스모크 판정 기준으로 사용한다. 각 단계에서 “6.0이 나오는가”를 확인한다.

레지스트리 연결에 대한 표기

운영이라면 9장의 등록과 이 장의 서버가 **공용 추적 서버 하나**를 바라본다. 실습은 서버 상주 없이 장별 로컬 SQLite로 축소했으므로, 서버 쪽 스크립트(bootstrap)가 9장과 같은 데이터·같은 절차로 chapter10 로컬 레지스트리를 재구성한 뒤 별칭을 소비한다. “같은 데이터”는 주장이 아니라

실측이다 — 훈련 데이터 지문 96f6b6b3ce9dcdd1... 이 9장 기록과 일치하지 않으면 bootstrap이 그 자리에서 실패하도록 만들었다(지문을 계약으로 쓰는 5-7-9장 기법의 재사용). 축소된 것은 서버의 위치이지, “서빙은 별칭만 소비한다”는 규율이 아니다.

공공 사례

콜센터 상담 지원 시스템이 “예측 근거를 소명하라”는 민원을 받았다고 가정한다. /predict 응답에 모델 신원(이름·버전)이 실려 있으면, 그 시각 그 응답을 만든 모델이 특정되고 — 9장 레지스트리를 따라 run·데이터 지문·파라미터까지 소급된다. 응답에 예측값만 있으면 소급은 “그때 아마 이 버전이었을 것”이라는 추정이 된다. 응답에 신원을 싣는 한 줄이 감사 가능성의 분기점이다. ### 핵심 정리 - 서빙 API의 최소 계약: 신원 응답(/health), 신원이 실린 예측(/predict), 스키마 검증(422) — 실측 전부 확인. - 별칭 로드 한 줄이 9장 레지스트리와 서빙의 연결 전부다 — 승격(별칭 이동) 때 서빙 코드는 바뀌지 않는다. - 알려진 입력 → 알려진 출력(9→6.0)이 파이프라인 전 층의 공통 스모크 기준이 된다.

10.4 Docker 이미지와 기본 테스트

학습 목표

이미지=코드+의존성+모델의 불변 스냅샷이라는 관점과, 레지스트리 참조/이미지 고정
이중 구조를 안다.
테스트를 층으로 쌓는 이유를 실제 겪은 오류로 확인한다.

직관: 실행 환경 차이를 제거한다

호스트에서 실행되는 서빙 앱은 그 호스트의 Python·라이브러리·경로에 의존한다. 다른 서버에 옮기는 순간 그 기대가 전부 깨질 수 있다. Docker 이미지는 실행 환경 자체를 산출물로 만든다 — 코드, 의존성(버전 고정), 그리고 이 실습에서는 **모델 실물까지** 하나의 불변 단위로 포장한다(베이스 python:3.13-slim, 실측 278MB — 환경·시점에 따라 변동). 9.1의 재현 4요소 중 “환경”이 문서가 아니라 실행 가능한 실물로 고정되는 것이다.

이 “한 번 만들면 고치지 않고 통째로 교체한다”는 관점은 인프라 운영에서 불변 서버(immutable server)라 이름 붙은 원칙의 연장이다 — Kief Morris가 martinowler.com에 정리한 Immutable Server 개념의 서술대로, 일단 배포된 실행 단위는 수정하지 않고, 변경이 필요하면 새 실물을 처음부터 생성해 교체한다. 수작업으로 패치한 서버가 저마다 조금씩 달라지는 “설정 표류”를 원천 차단하는 규율이다. CD4ML은 이 불변성을 ML로 확장한다 — 재현 가능한 모델 학습과 모델 서빙을 CD 파이프라인의 구성요소로 두어, 배포되는 것이 “코드”만이 아니라 “코드+모델+환경의 한 덩어리”가 되게 한다. 이 실습이 모델 실물을 이미지에 포함시키는 것은 그 확장을 최소 형태로 구현한 것이다(적용 자격: 두 출처 모두 일반·민간 도메인의 개념·방법론 차용이며 공공 직접 선례는 아니다).

모델을 이미지에 넣는 선택은 트레이드오프다. 실습 앱은 같은 코드로 두 로드 방식을 지원한다.

표 10.4 레지스트리 참조와 이미지 고정

	레지스트리 참조(호스트 실행)	이미지 고정(컨테이너)
모델 로드	<code>models:/...@champion</code> — 로드 시점의 champion	<code>/app/model</code> — 빌드 시점의 champion 복사본
모델 교체	별칭 이동 + 재기동	새 이미지 빌드 + 배포
장점	교체가 가볍다(개발·실험에 적합)	이미지=완결된 배포 단위 — 롤백은 이전 이미지 재기동, 레지스트리 장애·네트워크와 무관(폐쇄망 포인트)
위험	기동 시점에 따라 다른 모델을 로드할 수 있음	빌드와 별칭 이동 사이의 시차 — 이미지가 어느 champion인지 출처 기록 필수
신원의 근거	레지스트리에 조회(실측 <code>model_source=registry-alias</code>)	빌드 시 동봉한 <code>export_info.json</code> (실측 <code>model_source=local-dir</code>)

빌드 절차가 이 표의 오른쪽 열을 만든다: bootstrap이 champion 실물을 레지스트리에서 내려받아 빌드 컨텍스트에 두고, 내려받은 모델을 **로드·예측까지 검증**한 뒤(쓰기 성공을 믿지 않는다 — 9장 교훈), 어느 별칭의 몇 번 버전이었는지를 `export_info.json` 으로 함께 동봉한다. 컨테이너의 `/health`가 답하는 신원(`model_version=1`)의 근거가 바로 이 동봉 파일이다 — 이미지에 자기 출처가 함께 포함된다.

민간 배포 파이프라인도 같은 문제를 다른 도구로 푼다 — 이미지 태그(tag)와 배포 메타데이터에 커밋 해시·빌드 번호를 기록해 “지금 도는 이미지가 어느 소스에서 나왔는가”에 답한다. 목적은 `export_info.json` 과 같고, 다른 것은 빈도다. 하루에 여러 번 배포하는 조직에서는 이 기록을 사람이 적을 수 없으니 파이프라인이 자동으로 남긴다. 공공에서는 이 기록에 폐쇄망 반입 심사 절차를 추가한다.

실제로 겪은 오류: 모든 사전 게이트를 통과한 코드가 컨테이너에서 즉시 실패했다

이 실습을 구성하며 발생한 오류의 경과다. 서빙 앱은 설정 기본값(레지스트리 DB 경로 등)을 자기 파일 위치에서 두 단계 위 디렉터리(`code/10-1-model-api/` → `chapter10/`) 기준으로 계산했다. pytest 통과, 호스트 uvicorn 실측 통과 — 그런데 컨테이너는 `/health`가 끝내 응답하지 않았다. 원인: 이미지 안에서 앱은 `/app/app.py` 에 놓이므로 “두 단계 위”가 존재하지 않았고, 경로 계산이 **import 시점에** `IndexError`로 실패했다. 컨테이너에서는 그 기본값이 사용되지도 않는데(환경변수가 모두 지정된다), 기본값의 **계산 자체**가 배치 가정을 전제하고 있었던 것이다.

관찰할 것은 수정(경로 계산의 안전화)보다 실패의 구조다. 첫째, pytest도 호스트 실행도 이 결함을 검출할 수 없었다 — 둘 다 저장소 배치 위에서 실행되기 때문에 가정이 항상 참이었고, 가정이 깨지는 유일한 곳이 컨테이너였다. **실행 환경이 달라야만 드러나는 결함이 존재하며, 그것이 컨테이너 스모크가 별도 게이트여야 하는 이유다**(10.2의 “확신은 배포 전에 완전해지지 않는다”의 실측 증거이기도 하다). 둘째, 진단을 늦춘 것은 오류가 아니라 **정리 옵션**이었다 — 중단된 컨테이너를 자동 삭제(`--rm`)하도록 기동했더니 크래시(crash) 로그까지 함께 삭제되었다. 실패를 진단하려면 실패의 기록이 남아야 한다: 실습 코드는 로그가 남도록 기동하고 정리는 명시적으로 하도록 수정했다. 셋째, 감시 루프가 “중단된 서버”와 “느린 기동”을 구분하지 못해 90초 타임아웃을 모두 대기했다 — 수정 후에는 프로세스 생존을 함께 확인해 즉시 실패한다. 4장 유실, 8장 빈 적재, 9장 로드 실패에 이은 이 장의 침묵 실패 유형은 “게이트를 전부 통과하고 배포 직전 층에서야 드러나는 결함”이다.

테스트를 층으로 쌓는다

이 오류가 테스트 구조를 정당화한다. 실습의 게이트는 세 층이고, 각 층이 잡는 결함이 다르다.

pytest(TestClient): 서버 프로세스 없이 앱 객체를 직접 호출 — 계약 위반(신원 누락, 검증 누락, 알려진 입력→알려진 출력 불일치)을 초 단위에 검출한다. 실측 4건 통과.

호스트 실행 스모크: 실제 프로세스·실제 HTTP — 기동 절차, 포트, 레지스트리 연결의 결함을 검출한다.

컨테이너 스모크: 실제 배포 환경 — 이미지 배치·의존성 누락·경로 가정의 결함을 검출한다(위 오류가 정확히 여기서 검출되었다).

층의 마지막 질문은 **환경 동등성**이다: 같은 입력에 호스트와 컨테이너가 같은 답을 내는가. 실측 — 호스트 6.0 = 컨테이너 6.0, 두 환경 모두 422 거절 동작 동일. 8장의 “온라인=오프라인 일치”가 피처의 이중 저장 검증이었다면, 이것은 실행 환경의 이중화 검증이다 — 같은 실물(모델)을 두 곳에 두었으면, 같음을 가정하지 말고 실측한다.

기동 대기는 얼마로 설정하는가

층마다 “정상 기동했는가”를 확인하려면 얼마나 대기할지를 정해야 한다. 실습의 감시 루프는 헬스체크를 90초까지 대기한다(`run_chapter10.py`의 `wait_health`). 이 값을 결정하는 요인은 두 방향에서 온다. 아래로는 기동에 실제로 걸리는 시간 — 이 앱은 모델 로드가 기동 시점에 한 번 일어나므로(10.3), 모델 로드와 의존성 임포트가 끝나는 시간보다 넉넉해야 한다. 위로는 실패를 감지하는 속도 — 값을 키울수록 중단된 서버를 대기하는 시간이 늘어난다. 두 요구가 반대 방향이라 값 하나로는 풀리지 않는다. 실습이 택한 해법은 값을 늘리는 대신 **판정을 하나 더 붙이는 것이다**: 프로세스·컨테이너의 생존을 함께 확인해, 중단되었으면 타임아웃 전에 즉시 실패한다. 앞의 오류에서 90초를 모두 대기한 것은 값이 틀려서가 아니라 “중단”과 “지연”을 구분하는 판정이 없어서였다. 운영의 헬스체크도 구조가 같다 — 확인 주기·응답 타임아웃·연속 실패 횟수를 함께 정해야, 한 번의 느린 응답으로 멀쩡한 서버를 내리는 일이 생기지 않는다.

공공 사례

폐쇄망(인터넷 차단 망) 배포가 흔한 공공 환경에서 “이미지 고정” 방식은 선택이 아니라 전제가 된다 — 운영망의 컨테이너는 외부 레지스트리도, 내부 추적 서버도 접근하지 못할 수 있다. 모델 의존성이 전부 동봉된 이미지와 그 출처 기록(export_info)은 폐쇄망 반입 심사에서 “이 산출물이 정확히 무엇인가”에 답하는 최소 단위다. 보충의 폐쇄망 패키징이 이 연장선에 있다. ### 핵심 정리 - 이미지=코드+의존성+모델의 불변 스냅샷 — 재현 4요소의 “환경”이 실행 가능한 실물이 되고, 이미지가 곧 롤백 단위가 된다. - 실측 교훈: 실행 환경이 달라야만 드러나는 결함이 있다 — 컨테이너 스모크는 생략 가능한 중복이 아니라 독립 게이트다. - 환경 동등성은 가정이 아니라 실측 대상이다: 호스트 6.0 = 컨테이너 6.0, 신원과 출처는 각 환경의 근거(레지스트리/동봉 파일)로 답한다. - 기동 대기 시간은 로드 시간에서 올라오고 실패 감지 속도에서 내려온다 — 값 하나로 풀지 말고 생존 확인 판정을 함께 붙인다(실습 90초 + 생존 확인).

10.5 GitHub Actions 기반 검증 흐름

학습 목표

CI가 “무엇을 새로 하는가”(아무것도)와 “무엇을 바꾸는가”(반복의 주체)를 구분한다.
같은 검증을 어느 층에 둘지 판단하고, 게이트를 늘릴 때 무엇을 대가로 내는지 안다.
워크플로 정의를 읽고, 배포 전 검수 체크리스트를 실측에서 도출한다.

직관: CI는 새 검증이 아니라 잊지 않는 검증이다

10.4까지 게이트는 이미 다 있다 — bootstrap(데이터 계약), pytest(서빙 계약), 이미지 빌드, 컨테이너 스모크. 문제는 이 게이트들을 **누가 매번 실행하는가**다. 사람은 급하면 건너뛰다 — “한 줄만 수정했다”는 판단이 사고의 전형적 경로이고, 이 장의 실제 오류에서 확인했듯 한 줄짜리 경로 계산도 배포를 실패시킨다. CI(지속적 통합)는 이 실행의 주체를 사람에서 저장소로 옮긴다: push가 곧 게이트 실행이며, 생략이라는 선택지가 구조적으로 사라진다.

CI라는 이름의 원뜻이 이 규율을 담고 있다. Martin Fowler의 정의에서 지속적 통합은 구성원들이 각자의 작업을 자주 통합하고 통합할 때마다 자동화된 빌드·테스트로 검증하는 실천을 말한다 — 핵심은 도구가 아니라 “자주 + 자동 검증”이라는 습관의 구조화다. CD4ML은 이 실천을 ML 파이프라인으로 확장해, 데이터·모델·코드를 함께 다루는 지속적 배포 오케스트레이션으로 서술한다 (적용 자격: 두 출처는 각각 일반 SW·민간 ML의 방법론 차용이며 공공 직접 선례가 아니다; GitHub Actions 공식 문서는 이 실습이 쓴 도구의 직접 선례다). 이 실습의 워크플로는 그 실천의 최소 형태 — 이미 있는 게이트들을 push라는 사건에 묶어 자동 반복시키는 것이다.

실습 워크플로(`.github/workflows/ch10-model-api.yml`)는 GitHub Actions의 최소 문법으로 이것을 선언한다 — 언제(`on` : chapter10 경로의 push), 어디서(`runs-on` : GitHub이 제공하는 러너), 무엇을(`steps`). `steps`가 로컬 실행과 **같은 명령**이라는 것이 요점이다.

CI 단계(워크플로 정의)	로컬 동일 단계 (run_chapter10.py)	잡는 결함
의존성 설치	venv + requirements	의존성 선언 누락
bootstrap 실행	시나리오 1	데이터 지문 불일치(9장과 다른 데이터)
pytest	시나리오 2	서빙 계약 위반
docker build	시나리오 4 전반	이미지 정의·포장 결함
컨테이너 스모크(6.0 확인)	시나리오 4 후반	배치·환경 가정 결함(10.4의 오류 유형)

표기: 이 장의 인용 수치는 모두 **로컬에서 위 단계를 실행한 실측**이다. 원격 러너의 실행은 저장소에 push되는 시점에 일어난다 — 워크플로 파일은 그 반복을 선언해 둔 것이고, 로컬과 원격이 같은 명령을 돌리므로 같은 게이트다(러너 환경 차이로 결과가 다르면 그것 자체가 10.4의 교훈이 다시 적용되는 발견이다).

게이트를 어디에 둘 것인가

워크플로를 쓸 때 실제로 내리는 결정은 “무엇을 검증하는가”보다 **같은 검증을 어느 층에 두는가**다. 위 표의 다섯 단계는 앞에서 뒤로 갈수록 비싸진다 — pytest는 초 단위에 끝나지만 docker build와 컨테이너 스모크는 이미지를 만들고 컨테이너를 띄우는 시간을 쓴다. 그렇다고 뒤쪽을 제외하면 10.4의 경로 가정 결함처럼 앞 층에서 원리적으로 드러나지 않는 결함이 배포까지 통과한다. 배치를 정할 때 물을 것이 셋이다. 첫째, **이 결함이 앞 층에서 드러날 수 있는가** — 드러날 수 있으면 앞으로 당긴다(계약 위반은 pytest에서 잡는다). 둘째, **뒤 층에서만 드러나는가** — 그러면 그 층에 게이트를 상설한다(배치·환경 가정은 컨테이너에서만 드러난다). 셋째, **놓쳤을 때의 대가가 얼마인가** — 되돌리기 쉬운 결함은 뒤로 미뤄도 되지만, 되돌리기 어려운 결함은 비용을 감수하고라도 앞에 배치한다.

게이트를 늘리면 배포는 느려진다. 이 대가는 실재한다 — 검증 단계가 길어질수록 한 번 고친 것을 내보내기까지 걸리는 시간이 늘고, 그러면 배포 빈도(deployment frequency, DORA의 표준 지표)가 감소한다. 민간에서는 이 균형을 “빨리 배포해 실험 결과를 반영하는 이득 대 장애 비용”으로 계산한다. 공공에서는 비교 대상의 한쪽이 다르다 — 검증을 건너뛰어 얻는 것은 담당자의 시간이지만, 잃는 것은 “이 변경은 어떤 검증을 거쳤는가”에 답할 근거다. 그래서 공공 배포에서 게이트를 줄이는 판단은 속도의 문제가 아니라 증거의 문제로 다뤄야 하고, 줄인다면 무엇을 왜 줄였는지가 검수 문서에 남아야 한다.

CI가 검출하지 못하는 것도 명시해 둔다. CI의 게이트는 **코드·포장의 결함**을 검출한다 — 모델의 예측 품질이 데이터 변화로 낮아 가는 것(드리프트)은 push 이벤트와 무관하게 일어나므로 CI가 아니라 모니터링(11장)의 일이고, 부하에서의 성능은 12장의 일이다. “CI 통과”는 “배포해도 됨”의 필요조건이지 충분조건이 아니다.

배포 전 검수 체크리스트(정본 산출물)

이 장의 실측에서 도출한 체크리스트다. 실습이 실행 결과에서 자동 생성하며 (ch10_release_checklist.json), 8항목 전부 실측 통과가 이 장의 완료 조건이었다.

표 10.5 배포 전 검수 체크리스트(실습 자동 생성, 실측 전항 통과)

#	항목	이 장의 근거
1	자동 테스트 통과	pytest 4건(서빙 계약)
2	서빙 신원 응답 — /health가 모델 이름·버전을 답하는가	표 10.3(model_version=1)
3	알려진 입력 → 알려진 출력 스모크	9 → 6.0(손 계산 기준)
4	잘못된 입력 거절	422(스키마 계약)
5	컨테이너 = 호스트 동일 예측 (환경 동등성)	6.0 = 6.0
6	모델 실물 이미지 고정 + 출처 동봉	export_info.json(표 10.4)
7	레지스트리 버전 보존(롤백 전제)	v1·v2 보존, challenger 별칭(9장 연결)
8	검증 파이프라인 정의(CI 워크플로)	ch10-model-api.yml

수작업으로 쓰는 문서가 아니라 판정 실물에서 생성된다는 점은 8장 피쳐 정의서·9장 승격 검수 기록과 같은 철학이다 — 문서가 실물에서 나오면 문서와 실물이 어긋날 수 없다.

공공 사례

“수정 후 재배포는 어떤 검증을 거치는가”는 공공 시스템 유지보수 계약의 표준 감사 질문이다. CI 워크플로 파일은 그 답의 실물이다 — 어떤 게이트가, 어떤 순서로, 어떤 기준(알려진 입력→알려진 출력)으로 매 변경마다 실행되는지가 저장소에 선언되어 있고, 실행 이력이 러너에 남는다.

“담당자가 신중히 테스트합니다”라는 답과 워크플로 파일 한 장의 차이가, 9장 레지스트리에서 본 “사람의 기억 vs 시스템의 기록” 차이의 배포판이다.

역방향: 이 장의 배포 규율을 민간 현장에 옮기면

10.2에서 민간의 트래픽 분할을 공공의 shadow로 옮겼다. 반대 방향도 성립한다 — 이 장에서 세운 서빙·포장·검증의 규율을 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

이커머스 추천 모델 교체에서는 공공에서 shadow가 놓였던 자리에 카나리가 들어간다. 새 추천 모델을 전체 사용자의 1~5%에 먼저 노출하고, 그 집단의 클릭률·전환율이 대조군보다 통계적으로 유의하게 높으면 비율을 10%, 50%, 100%로 올린다. 지표가 저하되면 비율을 0으로 되돌린다 — 표 10.2의 “비율 0으로 축소”가 실시간으로 작동하는 형태다. 바뀐 것은 전략이지 판단 구조가 아니다. 여기서도 롤백 트리거는 지표·임계·관측 창 세 요소로 적어야 하고, 다만 지표 자리에 형평성 대신 매출 지표가 들어간다.

핀테크 결제 API 교체는 blue-green이 높은 비용을 감수할 만한 이유를 보여 준다. 구환경(blue)과 신환경(green)을 나란히 기동하고, 로드 밸런서(load balancer)의 타겟 그룹(target group)을 전환해 트래픽을 한 번에 넘긴다. 문제가 생기면 스위치를 되돌려 분 단위로 복귀한다. 이 장에서 “이전 이미지를 다시 기동하면 롤백”이라 적은 것이 여기서는 “스위치를 되돌리면 롤백”이 된다 — 롤백 목적지를 살려 두는 것이 롤백의 전제라는 성질은 같고, 살려 두는 대상이 이미지에서 환경 한 별로 커졌을 뿐이다. 대가도 정직하게 보인다: 결제가 몇 분 멈추는 비용이 환경 2벌의 상시 비용보다 크기 때문에 그 구조를 택한 것이다.

롤백 자동화는 이 장이 사람에게 남겨 둔 판단을 기계로 넘긴 경우다. 대규모 플랫폼은 배포 후 일정 시간 동안 오류율·응답 지연을 자동 감시하다가 임계를 넘으면 사람 개입 없이 이전 버전으로 되돌린다. 앞 절의 “CI 통과는 필요조건이지 충분조건이 아니다”가 이 장치의 존재 이유다 — CI가 못 잡는 것은 운영 환경에서만 드러나므로, 배포 직후의 감시가 마지막 게이트가 된다. 공공에서 같은 감시를 걸더라도 되돌림의 방아쇠를 자동으로 할지 사람이 당길지는 별도 결정이며, 어느 쪽이든 되돌린 사실과 근거가 기록으로 남아야 한다는 점이 다르다.

세 현장을 관통하는 것은 하나다. **배치·온라인의 선택, 배포 전략, 롤백 트리거와 목적지, CI 게이트의 배치** — 이 네 결정은 어느 현장에서도 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 형평성·감사·법적 의무라면, 민간에서는 매출·비용·SLA다. 방향은 여기서도 비대칭이다. 민간 설계를 공공에 도입하면 증거와 승인 이력이 한 겹 추가되지만, 공공 설계를 민간에 도입하면 그 층을 제외해도 손실이 없다 — 신원을 답하는 /health는 장애 대응 시간을 줄이고, 출처를 동봉한 이미지는 사고 원인 추적의 출발점이 되며, 자동 생성되는 검수 체크리스트는 배포 빈도를 올리면서도 변경 실패율을 눌러 두는 장치가 된다.

핵심 정리

CI는 새 검증을 만들지 않는다 — 있는 게이트의 실행 주체를 사람에서 저장소로 옮겨 “생략”을 구조적으로 제거한다.

워크플로의 단계는 로컬 실행과 같은 명령이다 — 로컬 실측이 곧 CI 게이트의 실측이고, 원격 실행은 push 시점에 반복된다.

게이트 배치는 세 질문으로 정한다: 앞 층에서 드러나는가, 뒤 층에서만 드러나는가, 놓쳤을 때의 대가가 얼마인가.

배포 전 검수 체크리스트는 실행 결과에서 자동 생성된다(실측 8항목 전항 통과) — CI 통과는 필요조건이지 충분조건이 아니다.

배치·온라인 선택, 배포 전략, 롤백 트리거와 목적지, 게이트 배치라는 네 결정은 도메인과 무관하게 내려야 하고, **근거만 도메인에 따라 달라진다**.

실습 10.1 모델 API 컨테이너 빌드와 호출(★★★★☆)

이 실습은 **설계** → **실행** → **대조** → **위험 식별** 네 단계로 진행된다. 순서에 의도가 있다. 이미 정해진 배포 방식으로 명령부터 실행하면 왜 shadow인지, 무엇이 저하되면 되돌리는지를 묻지 않게 되고, 그러면 실행 로그는 “근거 없이 통과한 게이트”로 남는다. 먼저 자기 판단으로 배포 설계를 정하고 결과를 예측한 다음 실행 로그와 대조한다. 검수·감사는 그 뒤 4단계에서 다룬다 — 설계 근거와 실행 증거가 모두 확보되어야 검수 질문에 답할 수 있기 때문이다.

실습 목표

요구사항에서 배포 전략·롤백 트리거·게이트 배치·기동 대기 시간을 스스로 정하고 근거를 적는다(설계).

9장 champion 별칭을 FastAPI 서빙이 로드하고, shadow(challenger)를 로그로 관찰한다.

같은 API를 pytest → 호스트 → 컨테이너의 층으로 검증하고, 환경 동등성(같은 입력→같은 예측)을 실측한다.

환경 동등성을 실행 전에 예측하고 실측과 대조해, 어긋난 지점에서 무엇을 놓쳤는지 설명한다.

입력 데이터

훈련 입력은 8장 피쳐 원천 스냅샷(9장과 동일한 `complaint_daily_features.parquet`, 원천은 4~7장 시뮬레이션 민원의 확정 집계)이다. 운영이라면 9장의 레지스트리를 공용 추적 서버로 직접 바라보겠지만, 실습은 장별 로컬 SQLite로 축소했으므로 bootstrap이 같은 데이터·같은 절차로 레지스트리를 재구성한다 — 같은 데이터임은 지문 `96f6b6b3ce9dcdd1...` (9장 기록과 일치)을 코드가 강제 확인한다. 성능 수치가 모델 품질의 주장이 아니라는 9장의 표기도 그대로 유효하다 — 이 장의 관찰 대상은 서빙·포장·검증의 **배포 규율**이다.

1단계 설계 — 배포 설계표와 환경 동등성 예측

명령을 실행하기 전에 배포 설계를 확정한다. 아래 요구사항을 읽고 **표 10.6의 빈칸을 직접 채운다**. 본문의 표 10.2·10.5는 완성된 예시이므로 그대로 옮겨 적지 않는다 — 채운 뒤에 비교한다.

요구사항(가상 발주 조건) - 자치구 담당자 화면 뒤에 일일 민원 건수 예측 API를 놓는다. 이용자는 25개 자치구의 담당 공무원이고, 예측값은 인력 배치 참고 자료로 사용된다. - 새 후보 모델(9장 v2)이 있고, 검수자는 승인 전에 실요청 위에서 champion과 비교하고 싶어 한다. - 발주처 요구: 같은 시각에 담당자마다 다른 모델이 응답하는 상태를 만들지 않는다. - 운영망은 폐쇄망이고, 이미지는 반입 심사를 거쳐 들어간다. 반입 후 문제가 나면 되돌릴 수 있어야 한다. - 담당자가 코드 한두 줄을 고쳐 재배포하는 일이 잦다.

표 10.6 배포 설계표 — 학습자 작성용

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
배포 전략(일괄·blue-green·카나리·shadow)		
롤백 트리거 — 지표·임계·관측 창		
롤백 목적지		
CI 게이트에 넣을 단계와 순서		
기동 대기(헬스체크) 타임아웃		
이 설계가 깨지는 조건		

작성할 때 지킬 것이 셋이다. 첫째, 근거 칸에 “기본값이라서” 또는 “보통 그렇게 한다”를 쓰지 않는다. 둘째, 롤백 트리거는 지표 하나만 적지 말고 임계와 관측 창을 함께 적는다(10.2) — 셋이 다 있어야 판정이 된다. 셋째, “이 설계가 깨지는 조건”에는 자기 선택이 불리해지는 상황을 적는다. 예를 들어 shadow를 선택했다면 계산 자원이 2배로 드는 상황이, 게이트를 네 개 모두 포함했다면 한 줄 수정에도 매번 이미지를 빌드하게 되는 상황이 여기에 해당한다.

환경 동등성 예측: 실행 스크립트는 같은 서빙 코드를 호스트(포트 8010)와 컨테이너(포트 8011)에서 각각 기동하고 같은 입력을 전송한다. 실행하기 **전에** 아래를 적어 둔다.

질문	내 예측	실제(3단계에서 기입)
두 환경의 예측값이 같겠는가		
다르다면 무엇 때문이겠는가		
/health의 <code>model_source</code> 가 두 환경에서 같겠는가		

예측 근거도 한 줄 적는다. 같은 코드가 두 환경에서 같은 답을 산출하려면 무엇이 함께 고정되어야 하는지를 검토한다.

2단계 실행 — 선수 조건과 실행 명령

Python 3.10 이상(3.13에서 검증)과, 시나리오 4를 위해 Docker 데몬(28.x에서 검증)이 필요하다. 실습은 증화되어 있어 Docker가 없으면 `--skip-docker` 로 시나리오 1~3까지 완주할 수 있다 (마지막 줄이 `CH10_RUN_PARTIAL` 로 표기된다 — 어디까지 검증했는지를 산출물에 그대로 기록하는 장치다).

```
cd practice/chapter10
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter10.py           # Docker 없이: python run_chapter10.py --skip-docker
```

시나리오	내용	대응 절
1	bootstrap — 레지스트리 재구성 (지문 확인), v1=champion-v2=challenger, champion export	10.3, 10.4
2	pytest — TestClient로 서빙 계약 4건 검증(서버 프로세스 없이)	10.3, 10.4
3	호스트 uvicorn(8010) — /health-/predict-422 실측, shadow 로그 관찰	10.2, 10.3
4	docker build → run(8011) → 동일 호출 → 환경 동등성 확인 → 정리	10.4, 10.5

핵심 코드

컨테이너 신원의 근거 — 빌드 시 동봉된 출처 파일을 /health가 읽는다(표 10.4의 오른쪽 열).

```
model = mlflow.pyfunc.load_model(uri) # 컨테이너: uri="/app/model"(이미지 고정)
info = json.loads(info_path.read_text(encoding="utf-8")) # 빌드 시 동봉한 export_info.json
return model, {"model_name": info["model_name"],
               "model_version": str(info["champion_version"]), "model_source": "local-dir"}
```

전체 코드는 *practice/chapter10/code/10-1-model-api/* 참고

(*app.py*, *bootstrap_registry.py*, *test_app.py*, *Dockerfile*), 오케스트레이션은

practice/chapter10/run_chapter10.py

실행 중 확인 체크리스트

- ☐ 시나리오 1에서 지문이 96f6b6b3ce9dcdd1... 로 출력되는가 — 9장과 같은 데이터라는 증명
- ☐ bootstrap 마지막에 export 로드 검증 예측 6.0이 나오는가 — 내려받은 실물의 로드까지 확인 (9장 교훈)
- ☐ 시나리오 2에서 4 passed인가
- ☐ 시나리오 3 health에 model_version=1, model_source=registry-alias, shadow_enabled=true가 기록되는가
- ☐ predict(전일 9) → 6.0, 잘못된 입력 → 422인가
- ☐ shadow 관찰에 challenger_forecast=7.3636이 있는가 — 표 9.2 계수로 손 계산(81/11)
- ☐ 시나리오 4 health의 model_source=local-dir이고, 예측이 호스트와 같은 6.0인가
- ☐ 마지막 줄이 CH10_RUN_PASS 인가

산출물 안내

산출물	내용	용도
ch10_deploy_report.json	bootstrap·테스트·호스트·컨테이너·shadow 실측	이 장 인용 수치의 증거 파일
ch10_release_checklist.json	배포 전 검수 체크리스트(정본, 실측 자동 생성)	표 10.5의 원본
ch10_bootstrap_summary.json	재구성 판정(승격·반려·별칭)·export 출처	9장 판정과의 대조
ch10_shadow_log.jsonl	shadow 관찰 로그(요청별 champion/challenger)	10.2의 실물 — 로컬 생성물
mlflow.db · mlruns/ · model_export/	로컬 레지스트리·모델 실물·빌드 컨텍스트	재실행하면 다시 생성되므로 커밋 제외
.github/workflows/ch10-model-api.yml	CI 워크플로 정의(저장소 루트)	10.5의 실물
표 10.6 배포 설계표(직접 작성)	배포 전략·롤백 트리거·게이트 배치·기동 대기와 각 근거	검수 자산 — 3단계 대조 후 확정

실행 결과(실제)

실제 실행 콘솔 출력의 발췌다.

```

== 시나리오 1: 레지스트리 재구성(bootstrap) ==
[bootstrap] 훈련 쌍 6개, 지문 96f6b6b3ce9dcdd1...
[bootstrap] champion=v1, challenger=v2
[bootstrap] export 로드 검증 예측=6.0 (champion=v1)
== 시나리오 2: 서빙 계약 테스트(pytest) ==
4 passed, 1 warning in 4.10s
== 시나리오 3: 호스트 서빙 - models:/...@champion 로드 ==
health={'status': 'ok', 'model_name': 'complaint_daily_forecaster', 'model_version': '1',
        'model_source': 'registry-alias', 'shadow_enabled': True}
predict(전일 9) → 6.0 / 잘못된 입력 → 422
shadow 관찰: {'lawd_cd': '11680', 'x_prev_count': 9, 'champion_forecast': 6.0,
              'challenger_forecast': 7.3636}
== 시나리오 4: 컨테이너 빌드·기동·호출 ==
이미지 빌드 완료: ch10-model-api (278 MB - 환경·시점에 따라 변동)
health={'status': 'ok', ..., 'model_version': '1', 'model_source': 'local-dir',
        'shadow_enabled': False}
predict(전일 9) → 6.0 / 잘못된 입력 → 422
CH10_RUN_PASS

```

별칭 연결 실측: 서버가 로드한 것은 `models:/complaint_daily_forecaster@champion` — 응답 신원 v1, 예측 6.0이 9장 판정·검산과 일치한다.

환경 동등성 실측: 호스트(`registry-alias`)와 컨테이너(`local-dir`)가 같은 입력에 같은 예측 6.0, 같은 422 거절 — 신원의 근거만 다르다(레지스트리 조회 vs 동봉 출처 파일).

shadow 실측: 응답은 champion 6.0, 로그에는 challenger 7.3636 — 반려된 v2가 이용자 위험 0으로 재평가 선상에 있다.

증거 파일 3종은 재실행 시 **바이트 단위로 동일함**을 확인했다(7장 해시 동일·9장 증거 결정성의 10장판) — 컨테이너까지 포함한 파이프라인에서도 “무엇이 바뀌면 사고인가”의 경계가 유지된다.

관찰

“6.0이 나오는가”라는 한 질문이 네 층(`bootstrap export 검증` → `pytest` → 호스트 → 컨테이너)을 관통했다 — 손 검산 가능한 스모크 기준의 효과이며, 6쌍 극소 데이터 설계의 이점이 배포 장에서도 그대로 나타난 것이다.

층마다 검출하는 결함이 다름을 실제 오류가 증명했다: 경로 가정 결함은 `pytest·호스트`를 통과하고 컨테이너에서만 실패했다(10.4). 층을 생략했다면 이 결함은 “배포 후 장애”로 발견됐을 것이다.

같은 서버 코드도 두 신원 근거(레지스트리/동봉 파일)로 같은 답을 산출했다 — 개발의 유연함(별칭 참조)과 배포의 불변성(이미지 고정)은 코드 분기가 아니라 로드 URI 하나의 차이이다.

이 장 역시 모델 코드보다 검증·포장 코드가 훨씬 길다 — 9장 관찰(훈련 2줄, 규율이 나머지)이 배포에서도 반복된다.

3단계 대조 — 예측과 실측의 대조

1단계 예측표의 “실제” 칸을 `ch10_deploy_report.json` 의 `consistency·host·container` 값으로 채운다.

질문	실측
두 환경의 예측값	호스트 6.0 = 컨테이너 6.0
<code>/health</code> 의 <code>model_source</code>	호스트 <code>registry-alias</code> / 컨테이너 <code>local-dir</code>
잘못된 입력	두 환경 모두 422

확인할 것이 셋이다.

예측을 “같다”로 적었다면 근거가 무엇이었는가. 값이 같은 것은 우연이 아니라 세 가지를 함께 고정했기 때문이다 — 모델 실물(이미지에 포함), 의존성 버전(`requirements.txt` 고정), 입력 계약(`pydantic` 스키마). 셋 중 하나만 달라져도 값은 달라질 수 있다. “같은 코드니까 같겠지”는 근거가 아니다. 코드는 세 요소 중 하나일 뿐이다.

예측을 “다르다”로 적었다면 무엇을 걱정했는가. 그 걱정이 곧 실제 위험 목록이다. 이미지와 호스트의 의존성 버전이 다른 경우, 빌드 시점의 champion과 현재 별칭이 가리키는 버전이 다른 경우(표 10.4의 “시차”) — 둘 다 실제로 일어나는 사고이고, 이번 실행에서 나타나지 않은 이유는 그것을 막는 장치를 넣어 두었기 때문이다.

`model_source` 가 다르다는 것은 무엇을 뜻하는가. 두 환경은 같은 답을 산출했지만 그 답의 출처를 서로 다른 곳에서 증명한다 — 호스트는 레지스트리에 조회하고, 컨테이너는 동봉된 `export_info.json` 을 읽는다. “같은 답이 나오는가”와 “무엇을 서빙 중인지 답할 수 있는가”는 별개 질문이며, 검수는 둘 다 묻는다.

여기서 놓치기 쉬운 것이 하나 더 있다. 환경 동등성은 **두 환경이 모두 실행되었을 때만** 확인할 수 있다. 10.4의 경로 가정 결함은 값이 달라지는 실패가 아니라 컨테이너가 응답하지 못하는 실패였다. 예측표를 값 비교로만 채웠다면 그 계층의 실패 유형을 모두 빠뜨린 것이다.

1단계 표 10.6으로 돌아가 “이 설계가 깨지는 조건” 칸을 다시 쓴다. 처음에 막연히 적었다면, 지금은 실측을 근거로 적을 수 있다. **제출하는 설계표는 이 수정을 거친 최종본이다.**

실습과 운영의 간극(축소 지점 목록)

축소 지점을 명시한다. 추적 서버(로컬 SQLite 재구성 ↔ 공용 서버를 등록·서빙이 공유), 트래픽(단발 호출 ↔ 동시 요청·부하는 12장), 배포 전략(shadow만 미니 구현 ↔ 카나리·blue-green은 라우팅 인프라 필요), CI(로컬 동일 단계 실측 ↔ push 시 원격 러너 반복), 이미지 배포(로컬 데몬 ↔ 이미지 레지스트리·서명·반입 심사), 인증·TLS(없음 ↔ 운영 필수). 축소된 것은 규모와 주변 체계이지 규율이 아니다 — 신원 응답, 스키마 검증, 알려진 입력→알려진 출력, 환경 동등성 실측, 출처 동봉은 어느 규모에서든 같은 형태다.

4단계 위험 식별 — 설계와 증거를 검수 언어로 옮기기

앞 세 단계에서 설계 근거(1단계)와 실행 증거(2·3단계)를 모두 확보했다. 검수·감사가 요구하는 것은 그 둘을 이어 붙인 형태다 — “무엇이 잘못될 수 있는가, 그렇게 판단한 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가”. 앞 네 행은 실행 증거에서, 마지막 행은 1단계 설계에서 나온다.

위험	실습에서의 근거	대응 방향
환경 가정 결함의 배포 유출	경로 계산 IndexError — 컨테이너 스모크에서만 발현	테스트 증화 + 컨테이너 게이트 상설
어느 모델이 서빙 중인지 불명	/health 신원 + export_info 출처 동봉	신원 없는 서빙 API를 검수에서 반려
검증 생략 배포	체크리스트 8항목 자동 생성 + CI 워크플로	게이트 실행 주체를 사람에서 저장소로
실패 잔해 소실로 진단 불가	-rm이 크래시 로그를 삭제(실제 경험)	로그 보존 기동 + 명시적 정리
되돌릴 기준 없이 배포	1단계 설계: 롤백 트리거의 임계·관측 창을 비운 채로도 실행이 끝난다	배포 문서에 지표·임계·관측 창 명시, 롤백 목적지(이전 이미지·이전 버전) 보존 확인

마지막 열이 “모니터링한다” 같은 말로 채워지면 그 행은 검수 문서에서 유용하지 않다. 대응 방향에는 **바꿀 설정값이나 바꿀 설계 결정**이 들어가야 한다.

제출물 3종: ① 표 10.6 배포 설계표(3단계 수정을 반영한 최종본), ② `ch10_deploy_report.json`, ③ 위 위험 식별 표. ①이 설계, ②가 증거, ③이 둘을 이은 검수 자산이다.

보충(전문가 트랙 포인터)

롤백 전략과 배포 승인 게이트: 이 장의 롤백 전제(버전 보존·이전 이미지)를 운영 절차로 확장한다 — 자동 롤백 조건(오류율 임계), 별칭 이동·배포의 승인 권한 분리, 배포 기록의 감사 로그화. 9장 보충(승인 워크플로)의 배포판이다.

이미지 보안 스캔과 secret 관리: 이미지는 불변이라 취약점도 불변으로 동봉된다 — 빌드 파이프라인의 이미지 스캔, 그리고 secret(API 키·DB 암호)을 이미지에 포함시키지 않고 주입하는 규율(1장 .env의 컨테이너판).

공공망 배포 제약과 폐쇄망 패키징: 10.4의 이미지 고정 방식이 기본기다 — 외부 저장소 접근 없는 빌드, 반입 심사용 SBOM(구성 명세), 오프라인 이미지 반입 절차.

핵심 용어

배치 추론 / 온라인 추론: 예측을 미리 만들어 저장 / 요청 순간 계산 — 입력 가용 시점과 대상 열거 가능성으로 선택

실패 반경(blast radius): 새 버전이 틀렸을 때 피해가 미치는 범위 — 배포 전략 비교의 축 일괄 / blue-green / 카나리 / shadow: 전량 교체 / 2벌+스위치 / 비율 점증 / 복제 관찰(반경 0)

champion / challenger: 서빙 중인 버전 / 실제 응답에는 사용하지 않고 로그로 평가 중인 후보(9장 별칭의 소비)

서빙 신원: /health-/predict 응답에 실리는 모델 이름·버전·출처 — “지금 운영 모델은?”의 API 답

lifespan(시작 시 로드): 모델 로드를 요청 경로 밖(기동 시점)으로 빼는 규율

스키마 계약(422): 선언된 입력 조건을 기계가 집행 — 잘못된 입력은 모델에 닿기 전에 거절
이미지 고정: 빌드 시점의 champion 실물을 이미지에 동봉 — 불변 배포 단위, 롤백은 이전 이미지

export_info(출처 동봉): 이미지가 어느 별칭·버전에서 나왔는지의 기록 — 이미지의 신원
환경 동등성: 같은 입력에 호스트와 컨테이너가 같은 답 — 가정하지 않고 실측한다

테스트 층화: pytest(계약) → 호스트(기동) → 컨테이너(배치·환경) — 층마다 잡는 결함이 다르다

CI(지속적 통합): 게이트 실행 주체를 사람에서 저장소로 — push마다 같은 검증 반복

워크플로: CI의 선언 파일(언제·어디서·무엇을) — 저장소에 커밋되는 검증의 실물

스모크 테스트: 알려진 입력 → 알려진 출력의 최소 확인 — 이 실습에서는 9 → 6.0

롤백 트리거: 어떤 지표가·어느 임계에서·얼마의 관측 창으로 나빠지면 되돌리는가 — 셋을 다 적어야 트리거다

게이트 배치: 같은 검증을 어느 층에 둘 것인가의 결정 — 앞 층은 싸고 좁게, 뒤 층은 비싸지만 배포 환경에 가깝다

검수 질문 한 줄 요약

발주·검수 문서에서 이 장은 세 질문으로 압축된다: ① 서빙 API가 자기 신원(모델 이름·버전·출처)을 스스로 답하는가(못 하면 운영 모델 불명 위험), ② 새 버전의 교체·롤백 절차가 실패 반경을 제한하는 전략(카나리·shadow 등)으로 문서화되어 있고 롤백 목적지(이전 버전·이전 이미지)가 보존되는가, ③ 변경마다 자동 실행되는 검증 파이프라인이 저장소에 선언되어 있는가(담당자의 성실성에 의존하면 검증 부재). 셋 다 없으면 그 시스템의 배포는 관리되는 절차가 아니라 개인의 손작업이다.

다음 장 예고

배포까지 끝났지만, 배포는 모델 생애의 시작이지 끝이 아니다. 서빙되는 모델은 시간이 지나며 조용히 낡는다 — 입력 데이터의 분포가 변하고(2장에서 본 드리프트가 운영 지표가 된다), 세상과 모델의 관계 자체가 변한다. CI는 push가 있어야 실행되지만, 드리프트는 push 없이 발생한다. 11장은 이 감시의 층을 다룬다: 시스템 지표(지연·처리량·오류율)와 모델 지표(예측 분포·성능)의 구분, 데이터 드리프트와 컨셉 드리프트, 그리고 재학습을 언제 트리거(trigger)할 것인가. 이 장의 shadow 로그(champion 6.0 vs challenger 7.3636)가 이미 그 감시 자료의 원형이다.

참고문헌

FastAPI. (n.d.). *FastAPI Documentation*. <https://fastapi.tiangolo.com/>

Uvicorn. (n.d.). *Uvicorn — An ASGI web server for Python*. <https://uvicorn.dev/>

MLflow. (n.d.). *MLflow Model Registry*. <https://mlflow.org/docs/latest/ml/model-registry/>

MLflow. (n.d.). *Deploy MLflow Models*. <https://mlflow.org/docs/latest/ml/deployment/>

Docker. (n.d.). *Docker Docs — Get Started / Dockerfile Reference*.
<https://docs.docker.com/get-started/>

GitHub. (n.d.). *GitHub Actions Documentation*. <https://docs.github.com/en/actions>

Fowler, M. (2010). *BlueGreenDeployment*.
<https://martinfowler.com/bliki/BlueGreenDeployment.html> (방법론 차용 — 일반 SW 배포, 공공 ML 직접 선례 아님)

Sato, D. (2014). *CanaryRelease*. [martinfowler.com](https://martinfowler.com/bliki/CanaryRelease.html).
<https://martinfowler.com/bliki/CanaryRelease.html> (방법론 차용 — 일반 SW 배포)

Google. (n.d.). *The Site Reliability Workbook, Ch. 16: Canarying Releases*.
<https://sre.google/workbook/canarying-releases/> (방법론 차용/배경 — SRE 일반)

Sato, D., Wider, A., & Windheuser, C. (2019). *Continuous Delivery for Machine Learning*.
[martinfowler.com](https://martinfowler.com/articles/cd4ml.html). <https://martinfowler.com/articles/cd4ml.html> (방법론 차용 — 민간 ML, 공공 직접 선례 아님)

Fowler, M. (2024). *Continuous Integration*.
<https://martinfowler.com/articles/continuousIntegration.html> (방법론 차용/배경 — 일반 SW)

Morris, K. (2013). *Immutable Server*. [martinfowler.com](https://martinfowler.com/bliki/ImmutableServer.html).
<https://martinfowler.com/bliki/ImmutableServer.html> (배경 — 인프라 불변성 개념)